

Linux distributions. This further spurred the growth of the GNU/Linux community.

Some of the earlier large-scale users include Amazon, the US Post Office and the German Army. It is interesting to note that Linux machine clusters were used while making movies like *Titanic* and *Shrek*. Today, we have Linux on gadgets like PDAs, mobiles, embedded applications and even on wristwatches. The introduction of 3D acceleration support, support for USB devices, single-click updates of systems and packages made Linux more popular. Desktop users can now log in graphically and start all applications without typing a single character on a terminal. At the same time, you still have the ability to access the core of the system.

Here are a few other utilities and programs (mostly from GNU) that added power to Linux:

- Bash: The GNU shell
- GCC: The GNU C compiler
- coreutils: A set of basic UNIX-style utilities
- findutils: For searching and finding files
- GDB: The GNU debugger
- fontutils: For converting fonts from one format to another or making new fonts
- Emacs: A very powerful editor
- Ghostscript and Ghostview: An interpreter and graphical front-end for the PostScript files
- GNU Photo: A software to manipulate the inputs from digital cameras
- Octave: To perform numerical computations
- GNOME and KDE: GUI desktop environments
- GNU SQL: A relational database system
- Radius: A remote authentication and accounting server

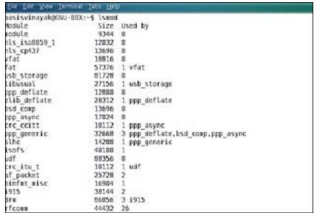
An introduction to the tech side

The Linux kernel is monolithic with module support. Beginners may not fully appreciate the meaning of this statement. A monolithic kernel is essentially a single large complex “do-it-yourself” kernel program, which has several different logical entities, and these run in kernel mode. The Linux kernel has a modular design. When you boot the system, only a minimal resident kernel is loaded into memory. When you request a feature that is not in the kernel, a kernel module (or driver) is dynamically loaded into memory. The object code normally has a collection of functions that implements a file system, a device driver or any other feature at the kernel’s upper layer. This has the following advantages: minimal main memory usage, modularised structure and platform independence.

Our first module

Let’s begin the technical side by writing a module:

```
#include <linux/module.h>
MODULE_AUTHOR("Aasis Vinayak PG");
```



Module	Size	Used By
module	9344 B	
lis_isc0059_1	12832 B	
lis_cp427	12608 B	
vfat	10816 B	
fat	57376 B	vfat
usb_storage	81728 B	
libusb	27756 B	usb_storage
zpp_deflate	12800 B	
zlib_deflate	26312 B	zpp_deflate
bsd_comp	13696 B	
zpp_async	17824 B	
zfc_cexit	10112 B	zpp_async
zpp_generic	32640 B	zpp_deflate,bsd_comp,zpp_async
libc	14208 B	zpp_generic
iso9660	40160 B	
udf	89256 B	
zfc_ata_t	10112 B	udf
ud_packet	25728 B	
libfat_ata	16960 B	
zfs	30144 B	
zfs	80856 B	zfs
zfs	44432 B	

Figure 1: A typical `lsmod` output

```
MODULE_LICENSE("GPL GPLv3");
static int new__init(void)
{
    printk("Let's begin our new segment!\n");
    return 0;
}
static void goodbye__old(void)
{
    printk("Adieu to segment 2!\n");
}
module_init(new__init);
module_exit(goodbye__old);
```

We have written a module. Let’s save this as `module1.c` in the home directory. Now we need a `make` file:

```
obj-m += module.o
all:
    make -C /lib/modules/$(shell uname -r)/build/ M=$(PWD) modules
clean:
    make -C /lib/modules/$(shell uname -r)/build/ M=$(PWD) clean
clean-files := Module.symvers
```

Before proceeding, check the kernel version you are using:

```
aasisvinayak@GNU-BOX:~$ uname -r
2.6.27-7-generic
```

Now we can make our module:

```
aasisvinayak@GNU-BOX:~$ make -C /lib/modules/$(uname -r)/build/
M="pwd"
make: Entering directory `/usr/src/linux-headers-2.6.27-7-generic/
CC [M] /home/aasisvinayak/module1.o
Building modules, stage 2.
MODPOST 1 modules
CC [M] /home/aasisvinayak/module1.mod.o
LD [M] /home/aasisvinayak/module1.ko
make: Leaving directory `/usr/src/linux-headers-2.6.27-7-generic/
```

You may change the directory by changing the `pwd`